VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
UNIVERSITY OF INFORMATION TECHNOLOGY
FACULTY OF SOFTWARE ENGINEERING



OBJECT-ORIENTED PROGRAMMING

LAB EXERCISE 4

Ho Chi Minh City, 11/2026

PRACTICAL CONTENT

SUBMISSION REQUIREMENTS AND INSTRUCTIONS

Submission instructions:

– Submit one .PDF report named STUDENTID_LAB4.PDF (e.g., 23520001_LAB4.PDF).

– Use the report template provided on Courses. Include source-code screenshots, console-output screenshots, and a short explanation of each important method (input, output, approach).

– Upload the complete Java source code to Drive and insert an accessible sharing link in the report.

– Implement each exercise as an independent Java console program. Students may use arrays or ArrayList and Scanner; a graphical interface or database is not required.

Note: The five exercises are practice tasks for the Lab 5 practical examination. Implement and test the programs independently; no sample solutions are provided.

LAB 4: COMPREHENSIVE OOP REVIEW – PREPARATION FOR LAB 5

This review integrates the concepts studied in Labs 1–3: Java control flow and arrays, classes and objects, constructors and overloading, encapsulation, inheritance, static members, and real-world management problems. Lab 4 explicitly consolidates the four OOP pillars: encapsulation, inheritance, polymorphism, and abstraction.

1. Learning Outcomes

– Identify a suitable superclass, two or more subclasses, and their responsibilities from a problem statement.

– Use private fields, controlled constructors, getters/setters and input validation.

– Apply inheritance with extends, super(...) and shared behavior.

– Use abstract classes or abstract methods to express common contracts; override behavior in each subclass.

– Store different concrete objects in one superclass-typed collection and invoke overridden methods polymorphically.

– Implement menu-based input, calculations, searches, statistics and boundary-case testing.

2. Quick Review: Four OOP Pillars

Pillar	What students must demonstrate
Encapsulation	private fields; safe constructors; validation; getters or behavior methods
Inheritance	meaningful IS-A relationship; reuse of shared data and methods
Polymorphism	overridden methods called through a parent-class reference/array/list
Abstraction	abstract superclass with at least one abstract operation implemented by subclasses

Minimal pattern (illustration only):

```
abstract class Item {
    private String id;
    public Item(String id) { this.id = id; }
    public String getId() { return id; }
    public abstract double calculateCost();
}
class FixedItem extends Item {
```

```
    private double cost;
    public FixedItem(String id, double cost) { super(id); this.cost = cost; }
    @Override public double calculateCost() { return cost; }
}
// Item[] items = { new FixedItem("I01", 100) };
// items[0].calculateCost();
```

Important: Method overloading (same name, different parameters) is different from method overriding (subclass changes an inherited method). Dynamic polymorphism in these exercises must be shown using overriding and a superclass-typed reference.

3. General Implementation Rules

– Each exercise must include an abstract base class, at least two concrete subclasses, and at least one @Override method.

– Declare object state private. Keep IDs non-empty, numerical amounts non-negative, and quantities/durations positive unless stated otherwise. Reject or re-enter invalid input without corrupting stored data.

– Provide at least a parameterized constructor in each class. Include one reasonable example of constructor or method overloading somewhere in the exercise (indicate it in the report).

– In the main program, store mixed subclasses in a collection whose declared element type is the abstract superclass. Call the common abstract method without downcasting to choose the calculation formula.

– Use a console menu with at least: add, display all, find/search, calculate/report statistics, and exit. Empty collections and "not found" cases must be handled.

– You may use basic Java (Scanner, String, arrays, ArrayList, loops, if/else). Files, streams, databases, external libraries and GUI are not required.

4. EXERCISES

Exercise 1: Library Borrowing Fee Management

Problem: A library lends two types of materials. Every item has a unique ID, title, author, and the number of days borrowed. Each type has a different total borrowing fee.

Type	Additional data	Total fee (VND)
PrintedBook	Number of pages	5,000 × days + 10 × pages
Magazine	Issue number	3,000 × days + 2,000

Requirements:

– Create abstract LibraryItem and subclasses PrintedBook and Magazine. Define abstract calculateFee() and override a common displayInfo() method.

– Input N items of mixed types. Display each item and its fee by invoking calculateFee() on a LibraryItem reference.

– Search by item ID; count the number of each type; calculate the total borrowing revenue; find the item with the highest fee.

– Validate days >= 1, pages >= 1 and issue number >= 1. Reject duplicate item IDs.

Suggested test: PrintedBook (200 pages, 3 days) costs 17,000 VND; Magazine (issue 12, 3 days) costs 11,000 VND. The total is 28,000 VND.

Exercise 2: Employee Payroll Management

Problem: A company employs full-time and part-time staff. Common information includes employee ID, full name, and department. Salary calculation differs by employment type.

Type	Additional data	Monthly salary (VND)
FullTimeEmployee	Base salary, allowance, workdays	base salary × min(workdays, 26) / 26 + allowance
PartTimeEmployee	Hours worked, hourly rate	hours worked × hourly rate

Requirements:

– Create abstract Employee with abstract calculateSalary(). Use FullTimeEmployee and PartTimeEmployee subclasses with suitable constructors.

– Input a mixed employee list; display payroll, including employee ID, type, department, and computed salary.

– Find employees by department (case-insensitive); calculate the total payroll; find the employee with the highest salary.

– Overload a search method: findEmployee(String id) and findEmployee(String name, String department), or implement an equally meaningful overload.

– Validate base salary, allowance, hours and hourly rate >= 0; 0 <= workdays <= 31.

Suggested test: Full-time base 10,400,000, allowance 1,000,000, workdays 26 → 11,400,000 VND. Part-time 40 hours × 50,000 → 2,000,000 VND.

Exercise 3: Online Order Delivery Management

Problem: An online store processes Standard and Express deliveries. Each delivery has an order ID, customer name, distance (km), and parcel weight (kg).

Delivery	Total shipping fee (VND)
StandardDelivery	15,000 + 3,000 × distance + 2,000 × weight
ExpressDelivery	30,000 + 5,000 × distance + 4,000 × weight

Requirements:

– Define abstract Delivery with calculateShippingFee() and subclasses StandardDelivery and ExpressDelivery. Encapsulate all fields.

– Add orders to a mixed collection; display all shipping fees; search by order ID; report total shipping revenue.

– Count orders for each delivery type and print all orders whose fee exceeds a user-entered threshold.

– Introduce a static field to count successfully created Delivery objects; display this counter and explain why static belongs to the class.

– Validate distance > 0 and weight > 0. Clearly state the currency and units in the output.

Suggested test: Standard, 10 km, 2 kg → 49,000 VND; Express, 10 km, 2 kg → 88,000 VND. Total → 137,000 VND.

Exercise 4: Cinema Ticket Sales Management

Problem: A cinema sells Regular and VIP tickets. Each ticket has an ID, movie title, and quantity. Tickets have different unit prices and discount rules.

Type	Unit price	Discount rule
RegularTicket	80,000 VND	10% discount if quantity >= 5; otherwise 0%

VIPTicket	150,000 VND	15% discount if quantity >= 3; otherwise 0%

Requirements:

– Design abstract Ticket with abstract calculateTotalPrice(), overridden in RegularTicket and VIPTicket. Provide appropriate encapsulation and constructor chaining.

– Input mixed ticket purchases; display quantities, unit prices, discount amounts and final totals.

– Search purchases by movie title; calculate the total revenue; determine the movie title with the highest total revenue (aggregate all purchases of the same movie).

– Sort or display purchases from highest to lowest final total without relying on advanced frameworks.

– Validate quantity >= 1 and reject duplicate ticket IDs.

Suggested test: 5 Regular tickets → 360,000 VND; 3 VIP tickets → 382,500 VND; combined revenue → 742,500 VND.

Exercise 5: University Course Registration and Tuition

Problem: A university manages course registrations for two kinds of courses: Theory and Laboratory. Each course registration has a course code, course name, number of credits, and number of registered students. Different types use different tuition rates.

Type	Additional data	Tuition per student (VND)
TheoryCourse	No extra data	credits × 450,000
LabCourse	Equipment fee per student	credits × 600,000 + equipment fee

Requirements:

– Create abstract CourseRegistration with abstract tuitionPerStudent(); derive TheoryCourse and LabCourse. Implement common displayInfo() and override it where appropriate.

– Input a mixed list of courses. Display the tuition per student and the expected total collected for each course (tuition per student × registered students).

– Search by course code; find the course with the largest expected collection; compute total expected collection across all courses.

– Count Theory versus Laboratory courses, and calculate the average tuition per student across registered students (weighted by student counts); display N/A if total registered students = 0.

– Validate credits >= 1, registered students >= 0 and equipment fee >= 0. Reject duplicate course codes.

Suggested test: Theory (3 credits, 20 students) → 1,350,000 VND/student, 27,000,000 VND total; Lab (2 credits, equipment fee 100,000, 10 students) → 1,300,000 VND/student, 13,000,000 VND total. Combined collection → 40,000,000 VND; weighted average → 1,333,333.33 VND/student.

5. SELF-CHECK BEFORE LAB 5

– Can you explain where each of the four OOP pillars appears in your program, referring to specific classes and methods?

– Does a superclass reference call the correct subclass implementation at runtime?

– Are constructors, this/super, get/set methods and access modifiers used correctly?

– Can the program handle empty lists, duplicates, invalid numbers, boundary thresholds and searches with no result?

– Are the calculations consistent with the supplied formulas and sample tests?

– Can you create and run the full program independently without copying a solution?

6. SUGGESTED PRACTICE PLAN AND ASSESSMENT CHECKLIST

Recommended sequence: Exercise 1 (modeling and basic polymorphism), Exercise 2 (overloading and reporting), Exercise 3 (static members), Exercise 4 (aggregation and sorting), Exercise 5 (integrated exam-style practice). Each exercise is independent, not one large combined project.

Criterion	Suggested weight
Class design, abstraction and inheritance	25%
Encapsulation, constructors and validation	20%
Overriding and dynamic polymorphism	20%
Correct processing, statistics and menu functionality	25%
Test cases, readable code and report	10%

The percentages above are a suggested self-assessment rubric for Lab 4 practice, not an official Lab 5 examination grading scheme.